

GuardShield-Runtime: Runtime Action Guarding and Shielding for Progress-Preserving Dense-Traffic Driving

Anonymous Authors

Anonymous Affiliation

Abstract—Dense-traffic autonomous driving requires reinforcement learning policies to preserve route progress while avoiding collision and off-road failures. Reward-only safety can be misleading in this setting: a policy may reduce measured cost by barely moving rather than by driving safely. This paper presents GuardShield-Runtime, a runtime safety architecture that separates policy learning, action guarding, final shielding, and progress-gated risk shaping. The action guard corrects unsafe short-horizon commands before execution, while the shield branch provides a final braking fallback under severe time-to-collision or overspeed conditions. In MetaDrive dense traffic at density $d = 0.15$, the Guard runtime mode achieves 63.3% success, 20.7% episode-level safety cost, and 0.868 route completion. Removing the action guard collapses success to 0.0% and increases cost to 100.0%, identifying runtime guarding as the critical mechanism. External-seed evaluation over 300 episodes further shows that the Shield runtime mode obtains the highest mean success, although confidence intervals overlap with other competitive variants. These results support a simulator-bounded conclusion: runtime action intervention improves the progress-safety trade-off under the tested dense-traffic settings, while higher-density generalization remains open.

Index Terms—safe reinforcement learning, autonomous driving, action guard, shielding, progress-gated risk penalty, MetaDrive, dense traffic

I. Introduction

Reinforcement learning (RL) for autonomous driving must make progress along the route while keeping the vehicle out of collision and off-road states. Dense traffic makes this requirement difficult because the progress objective and the safety objective can fail in opposite directions. A cautious policy may reduce the measured cost by barely moving, whereas an aggressive policy may finish more routes by accepting unsafe interactions. A dense-traffic evaluation is therefore meaningful only when success, episode-level safety cost rate, route completion, collision, and off-road rates are interpreted together.

Reward-level safety alone is not sufficient for this setting. Safe RL methods often encode safety through constraints, costs, or risk-sensitive objectives, but a cost-minimizing driving policy can also create a non-motion artifact. It looks safe because it avoids traffic interaction, not because it drives safely. We therefore study whether safety intervention should be applied at the reward level, at the action-correction level, or at the final execution

level. This question shifts the paper away from ranking a single scalar objective and toward identifying which mechanism changes the dense-traffic failure mode.

This paper proposes GuardShield-Runtime, a progress-preserving runtime safety architecture for dense-traffic RL driving. The architecture separates a PPO policy, a runtime action guard, a final shield branch, and a progress-gated risk penalty. The policy first proposes an action, the guard corrects unsafe short-horizon longitudinal commands, the shield branch applies a stronger fallback under severe time-to-collision (TTC) or overspeed conditions, and the risk penalty is activated only when the vehicle is making measurable progress. This decomposition allows reward shaping, action guarding, and execution shielding to be evaluated as distinct intervention points.

The contributions are threefold. First, we introduce GuardShield-Runtime, a runtime safety architecture that separates policy learning, action guarding, final shielding, and progress-gated risk shaping. This separation makes it possible to study safety as both a learning signal and an execution-time intervention. Second, we provide a mechanism-level evaluation protocol for dense-traffic RL driving. Instead of ranking policies by a single reward or cost score, the protocol jointly reports success, episode-level safety cost, route completion, collision, and off-road rates to expose non-motion artifacts. Third, we show that runtime action guarding is the decisive component under the tested MetaDrive protocol. The Guard runtime mode gives the best held-out success-cost balance, the Shield runtime mode gives the highest external-seed mean success, and removing the guard causes a complete failure. Stress-density results further identify the remaining boundary rather than claiming solved generalization.

II. Related Work

A. Safe and Constrained Reinforcement Learning

Safe RL commonly encodes safety through constraints, costs, risk-sensitive objectives, or safe-exploration mechanisms [1]–[4]. Lagrangian and Lyapunov approaches optimise policies under cost constraints or stability-inspired conditions [5], [6], while continuous-action safe-exploration methods modify proposed actions to satisfy local safety conditions [7]. These methods are important for learning

safety-aware policies, but dense-traffic driving exposes a specific reward-level failure: reducing cost can coincide with low route progress. GuardShield-Runtime therefore treats progress-gated risk shaping as one intervention point and compares it against action-space guarding and execution-time shielding.

B. Runtime Shielding and Safety Filters

Runtime shielding and safety filters prevent unsafe actions before execution through formal constraints, reachability-style checks, or predictive safety filters [8]–[10]. Control-barrier-function methods similarly enforce safety-critical constraints in continuous control [11], [12]. GuardShield-Runtime follows this runtime-intervention philosophy, but its contribution is empirical and mechanism-level. We isolate action guarding, final shielding, and risk shaping in the same dense-traffic RL protocol to determine which intervention changes the progress–safety failure mode.

C. Closed-Loop Autonomous Driving Evaluation

Closed-loop driving benchmarks require evaluating route progress and safety jointly, because open-loop prediction or isolated reward values do not capture interaction failures. MetaDrive provides configurable traffic density and procedural scenario generation [13]; CARLA and highway-style environments similarly support closed-loop autonomous-driving evaluation under controlled scenarios [14], [15]. Deep RL has been widely studied for autonomous driving decision making [16]–[18], with PPO and SAC serving as common continuous-control backbones [19], [20]. Unlike work that reports a single final policy, this paper uses mechanism-level ablations to distinguish reward shaping, action guarding, and execution shielding. The non-motion issue is also related to reward hacking and specification gaming in AI systems [21].

III. Method

A. Runtime Decomposition

GuardShield-Runtime separates learning, runtime intervention, and risk shaping (Fig. 1). At each step, the PPO policy samples a raw two-dimensional MetaDrive action $a_t = (\delta_t, u_t)$, where δ_t is steering and u_t is the longitudinal command. The runtime safety layer maps this raw command to a guarded action, and the final shield branch can replace the longitudinal command with a braking fallback:

$$a_t \sim \pi_\theta(\cdot | o_t), \quad \tilde{a}_t = G(o_t, a_t), \quad a_t^{\text{exec}} = S(o_t, \tilde{a}_t). \quad (1)$$

Here, G is the action guard, S is the shield branch, $\tilde{a}_t$ is the corrected action, and a_t^{exec} is the executed action. In the implementation used for the reported experiments, guarding and shielding preserve the steering command and modify the longitudinal command when short-horizon TTC or speed predicates are violated.

B. TTC and Speed Guarding

The action guard estimates a forward TTC from observable nearby traffic. For each detected vehicle, the wrapper projects relative displacement and relative velocity onto the ego heading. A vehicle contributes to TTC only if it is in front of the ego vehicle, lies within 1.8 ego widths laterally, and has positive closing speed. The minimum TTC is

$$\text{TTC}_{\min} = \min_i \frac{\Delta x_i^\parallel}{v_{\text{ego},i}^\parallel - v_i^\parallel}, \quad (2)$$

computed over valid forward vehicles. The continuous TTC risk is

$$\rho_{\text{ttc}} = \text{clip} \left(\frac{\tau_{\text{ttc}} - \text{TTC}_{\min}}{\tau_{\text{ttc}}}, 0, 1 \right), \quad (3)$$

with $\tau_{\text{ttc}} = 12.0$ s in the reported protocol. The runtime thresholds are $\tau_{\text{soft}} = 12.0$ s and $\tau_{\text{hard}} = 7.2$ s. If $\text{TTC}_{\min} < \tau_{\text{soft}}$, the guard limits the longitudinal command to at most -0.6 . If $\text{TTC}_{\min} < \tau_{\text{hard}}$, the shield branch limits it to at most -1.0 . A separate speed predicate uses the target speed $v^* = 18$ km h⁻¹: the command is capped at 0 above v^* and at -0.35 above $v^* + 5$ km h⁻¹. These interventions are logged as soft, hard, and overspeed shield events.

C. Progress-Gated Risk Penalty

Risk shaping is used as a learning signal rather than as a safety guarantee. The route-progress gate prevents the policy from receiving risk-avoidance benefit while stationary:

$$r_t = r_t^{\text{task}} - \lambda \mathbb{I}(p_t > p_{\min}) \rho(o_t, a_t), \quad (4)$$

where p_t is one-step route-completion progress, $p_{\min} = 10^{-4}$ in the gated-risk extension, and $\mathbb{I}(\cdot)$ is an indicator. The risk score is implemented as

$$\rho(o_t, a_t) = w_{\text{ttc}} \rho_{\text{ttc}} + w_{\text{lane}} d_{\text{lane}}^2 + w_{\text{smooth}} |\Delta \delta_t| + w_{\text{accel}} |\Delta u_t| + w_{\text{speed}} \rho_{\text{speed}}^2 + w_{\text{cost}} c_t, \quad (5)$$

where d_{lane} is lane deviation normalised by half lane width, c_t is the MetaDrive safety cost, and $\rho_{\text{speed}} = \max(v_t - v^*, 0)/v^*$. For the Guard and Shield runtime modes, reward-level risk shaping is disabled and the runtime filter is the active safety mechanism. For the gated-risk extension, the penalty is active only when the vehicle is moving at least 3 km h⁻¹ or makes route progress above 10^{-4} and a TTC, lane, or speed risk is present.

The guard and shield differ in intervention strength. The guard is a corrective layer that attempts to preserve the policy command through soft braking. The shield branch is a conservative fallback for severe TTC or overspeed violations. Lane deviation is not used as a separate lane-centering fallback in this implementation; it enters the risk penalty and the evaluation metrics. This distinction is important for reproducibility because the reported runtime effect comes from TTC and speed action intervention, not from an unreported lane controller.

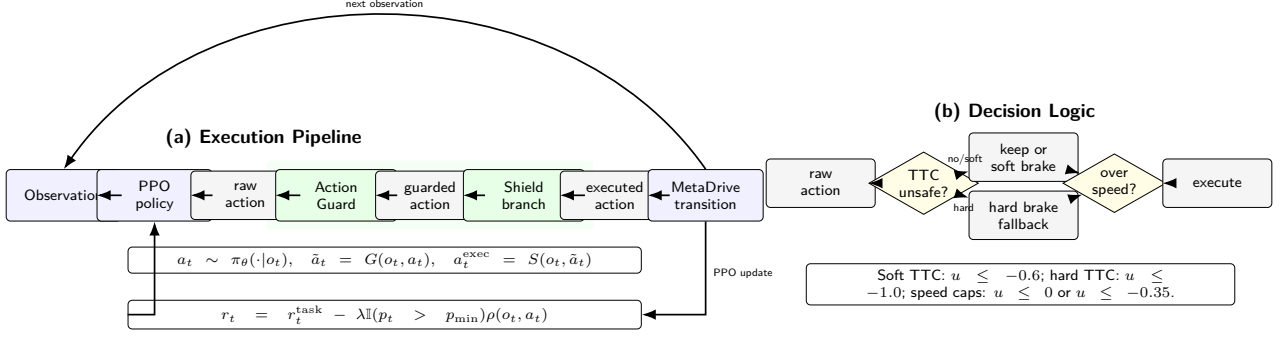


Fig. 1. Runtime action-intervention architecture of GuardShield-Runtime. The left panel shows the execution pipeline from observation to raw action, guarded action, shielded action, and MetaDrive transition. The bottom loop indicates that risk shaping is used for learning, not as a safety guarantee. The right panel shows the decision logic for TTC and speed predicates.

Algorithm 1 GuardShield-Runtime execution step

- 1: Observe o_t and sample $a_t = (\delta_t, u_t) \sim \pi_\theta(\cdot|o_t)$.
- 2: Compute $\text{TTC}_{\min}$ from detected forward vehicles and measure ego speed v_t .
- 3: Set $\tilde{a}_t \leftarrow a_t$.
- 4: if $\text{TTC}_{\min} < 7.2$ s then
- 5: $\tilde{u}_t \leftarrow \min(u_t, -1.0)$ ▷ hard shield fallback
- 6: else if $\text{TTC}_{\min} < 12.0$ s then
- 7: $\tilde{u}_t \leftarrow \min(u_t, -0.6)$ ▷ soft guard correction
- 8: end if
- 9: if $v_t > 23$ km h⁻¹ then
- 10: $\tilde{u}_t \leftarrow \min(\tilde{u}_t, -0.35)$.
- 11: else if $v_t > 18$ km h⁻¹ then
- 12: $\tilde{u}_t \leftarrow \min(\tilde{u}_t, 0)$.
- 13: end if
- 14: Execute $a_t^{\text{exec}} = (\delta_t, \tilde{u}_t)$ and log success, cost, route completion, collision, off-road, and intervention events.

IV. Experimental Protocol

A. Environment and Metrics

All experiments are conducted in MetaDrive dense-traffic scenarios with random traffic, random lane width, random lane number, 120 lidar rays, 60 m lidar range, and four tracked surrounding objects. Traffic density is varied across $d \in \{0.00, 0.08, 0.15, 0.20, 0.25\}$. We report success rate, episode-level safety cost rate, route completion, collision rate, and off-road rate. Safety cost is the episode-level MetaDrive cost and, in the reported tables, equals the union of collision and off-road failures. Route completion is reported as a continuous value in $[0, 1]$.

B. Training and Evaluation Splits

All competitive variants use the same PPO backbone, observation space, action space, and deterministic evaluation path. Unless otherwise stated, models are trained for 10^6 environment steps with 16 vectorised environments, horizon 1500, two hidden layers of width 256, learning rate

3×10^{-4} , entropy coefficient 0.01, $\gamma = 0.99$, GAE $\lambda = 0.95$, PPO clip 0.2, 1024 rollout steps, batch size 256, and 10 epochs per update. The training split uses 200 scenarios from start seed 0. The held-out evaluation split uses 100 scenarios from start seed 10000 and 50 episodes per density for each training seed. The primary tables report means over three training seeds. External-seed evaluation uses the same frozen models under additional scenario starts 20000 and 30000, giving six evaluation units and 300 episodes per variant at each reported density.

C. Compared Variants

Guard runtime refers to the three-seed ‘guard_only’ configuration: action guard on, reward-level risk penalty off, and curriculum on. Shield runtime refers to the ‘shield_only’ configuration: the same runtime TTC/speed filter is active, reward-level risk penalty is off, and the model is trained without curriculum progression. Full extension combines action intervention, curriculum, and risk shaping. Retuned full extension uses the same component family with the selected retuning. Gated-risk extension activates the progress-gated risk penalty. The ablations remove the TTC penalty, lane penalty, smoothness and acceleration penalties, or the runtime action guard from the guard-centred protocol. Baseline, curriculum-only, and risk-only controls are retained as lower-bound diagnostics because they expose unsafe driving or non-motion artifacts.

D. Statistical Reporting

Table I summarises how each evaluation block supports the claim. For binary rates such as success and cost, the external-seed block reports Wilson 95% confidence intervals [22]. Since intervals overlap, the Shield runtime mode is interpreted as the highest mean result rather than as a statistically isolated winner.

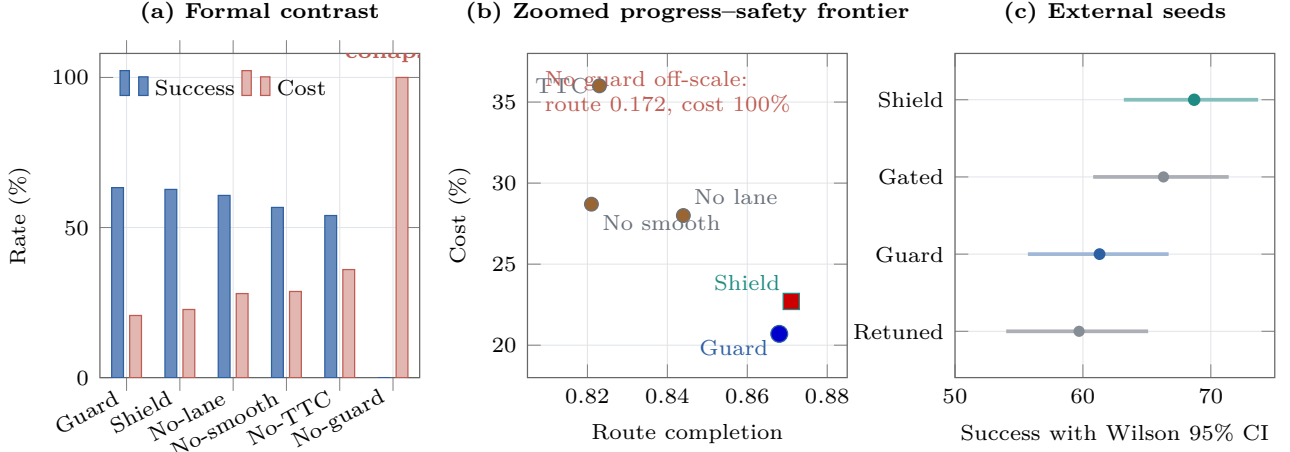


Fig. 2. Primary result summary. Panel (a) reports formal held-out success and cost for runtime modes and mechanism ablations. Panel (b) shows the progress-safety frontier, including each ablation and the no-guard off-scale collapse. Panel (c) reports external-seed Wilson intervals using 300 episodes per variant.

TABLE I
Evaluation blocks and their role in the claim.

Block	Purpose	Density	Output
Primary	success-cost comparison	0.15	Table II, Fig. 2
Density	normal-density trend	0.00/0.08/0.15	Table III
Ablation	mechanism necessity	0.15	Table IV
Robust/stress	seed robustness and boundary	0.15/0.20/0.25	Tables V, VI

V. Results

A. Primary Held-Out Comparison

Runtime action guarding provides the strongest held-out success-cost balance at $d = 0.15$. Table II and Fig. 2 show that Guard runtime achieves 63.3% success, 20.7% cost, and 0.868 route completion. Shield runtime is close on the same held-out slice and is tested again under external seeds. Removing the action guard directly isolates the critical runtime component, reducing success from 63.3% to 0.0% and increasing cost from 20.7% to 100.0%.

B. Normal-Density Generalisation

Increasing traffic density consistently reduces success and increases cost across all competitive variants. As shown in Table III, Guard and Shield remain on the leading frontier from $d = 0.00$ to $d = 0.15$, but neither removes the density dependence of the task. This result supports a bounded runtime-family claim rather than a general dense-traffic solution.

C. Mechanism Ablation

The ablation results identify action guarding as the critical mechanism. Table IV shows that reward-term removals degrade performance gradually, whereas removing the runtime action guard changes the failure mode

completely. The TTC, lane, and smoothness terms act as refinements around the guard-centred mechanism, but the no-guard row explains why runtime intervention is necessary.

D. External-Seed Robustness

External-seed evaluation tests whether the ranking is stable under additional scenario seeds. Table V and Fig. 2(c) show that Shield runtime obtains the highest mean success over 300 episodes at $d = 0.15$. Wilson intervals overlap, so this block supports a highest-mean robustness claim rather than a statistically isolated superiority claim.

E. Stress-Boundary Evaluation

Stress-density evaluation exposes the boundary of the current method. At $d = 0.20$ and $d = 0.25$, all variants degrade substantially. Table VI shows that Shield runtime obtains the best result at $d = 0.20$, while Guard runtime obtains the best result at $d = 0.25$. The absolute success rates show that high-density generalisation remains unsolved.

TABLE VI
Stress-boundary evaluation beyond the primary held-out setting. All variants degrade substantially, so this block identifies the remaining boundary rather than claiming solved high-density generalisation. Best values are shaded and bold; second-best values are underlined.

Variant	$d = 0.20$			$d = 0.25$		
	Succ. $\uparrow$	Cost $\downarrow$	Route $\uparrow$	Succ. $\uparrow$	Cost $\downarrow$	Route $\uparrow$
Shield runtime	48.0	36.0	0.778	19.3	58.7	0.609
Guard runtime	41.3	40.7	0.733	26.7	48.7	0.658
Gated-risk	44.7	36.7	0.766	22.0	52.7	0.628
Retuned full	45.3	38.0	0.753	17.3	68.7	0.571
Full extension	38.0	40.0	0.744	20.7	58.7	0.587

TABLE II

Primary held-out comparison at density $d = 0.15$. The no-guard row directly isolates the contribution of runtime action guarding. Best values are shaded and bold; second-best values are underlined.

Variant	Role	Succ. $\uparrow$	Cost $\downarrow$	Route $\uparrow$	Coll. $\downarrow$	Off-road $\downarrow$	Δ Succ.	Δ Cost
Guard runtime	main runtime mode	63.3	20.7	<u>0.868</u>	20.0	<u>0.7</u>	reference	reference
Shield runtime	shield runtime mode	<u>62.7</u>	<u>22.7</u>	0.871	<u>22.7</u>	0.0	-0.6	+2.0
w/o lane reward	reward ablation	60.7	28.0	0.844	28.0	0.0	-2.7	+7.3
w/o smoothness reward	reward ablation	56.7	28.7	0.821	28.0	<u>0.7</u>	-6.7	+8.0
w/o TTC reward	reward ablation	54.0	36.0	0.823	36.0	0.0	-9.3	+15.3
No action guard	guard ablation	0.0	100.0	0.172	51.3	48.7	-63.3	+79.3

TABLE III

Density sweep for competitive variants. The runtime guard and shield modes stay on the leading frontier from easy traffic to the formal held-out density. Best values are shaded and bold; second-best values are underlined.

Variant	$d = 0.00$			$d = 0.08$			$d = 0.15$		
	Succ. $\uparrow$	Cost $\downarrow$	Route $\uparrow$	Succ. $\uparrow$	Cost $\downarrow$	Route $\uparrow$	Succ. $\uparrow$	Cost $\downarrow$	Route $\uparrow$
Guard runtime	97.3	0.0	0.990	87.3	9.3	0.940	63.3	20.7	0.868
Shield runtime	99.3	0.0	0.992	91.3	6.7	0.959	<u>62.7</u>	<u>22.7</u>	0.871
Retuned full extension	96.7	0.0	<u>0.991</u>	<u>88.7</u>	<u>7.3</u>	0.953	62.0	24.0	0.853
Full extension	<u>97.3</u>	0.0	0.989	83.3	13.3	0.929	62.0	24.7	0.865
Gated-risk extension	94.0	0.0	0.988	86.7	8.0	0.961	60.0	25.3	0.849

Exact ties are marked together. Failure controls are not ranked here because they are diagnostic lower bounds rather than competitive density-sweep candidates.

TABLE IV

Mechanism ablation around the Guard runtime reference at $d = 0.15$. Deltas are measured relative to Guard runtime. Reward-term removals degrade the balance, while removing the action guard causes the decisive collapse.

Variant	Success $\uparrow$	Δ Succ.	Cost $\downarrow$	Δ Cost	Route $\uparrow$	Δ Route	Collision $\downarrow$	Interpretation
Guard runtime	63.3	+0.0	20.7	+0.0	0.868	+0.000	20.0	reference
w/o TTC reward	54.0	-9.3	36.0	+15.3	0.823	-0.045	36.0	TTC loss hurts safety
w/o lane reward	60.7	-2.7	28.0	+7.3	0.844	-0.024	28.0	lane term improves stability
w/o smoothness reward	56.7	-6.7	28.7	+8.0	0.821	-0.047	28.0	smoothness term improves control
No action guard	0.0	-63.3	100.0	+79.3	0.172	-0.695	51.3	guard removal collapses

F. Model-Selection Closure

Seed-level and parameter-sensitivity checks were used as closure tests. Several probes improve one metric while worsening another; for example, a lower-cost guard probe reduces success, and a higher-success probe raises cost. These checks are retained as diagnostics rather than main configurations because the paper’s claim is based on the joint progress–safety criterion.

TABLE V

External-seed robustness at $d = 0.15$. Wilson intervals are shown for the two primary binary outcomes, success and cost. Route completion is reported as the mean. Best values are shaded and bold; second-best values are underlined.

Rank	Variant	Episodes	Success $\uparrow$ [95% CI]	Cost $\downarrow$ [95% CI]	Route $\uparrow$	Collision $\downarrow$	Off-road $\downarrow$
1	Shield runtime	300	68.7 [63.2, 73.7]	20.7 [16.5, 25.6]	0.878	20.0	0.7
2	Gated-risk extension	300	<u>66.3</u> [60.8, 71.4]	<u>21.3</u> [17.1, 26.3]	<u>0.866</u>	<u>21.3</u>	0.0
3	Guard runtime	300	61.3 [55.7, 66.7]	24.0 [19.5, 29.1]	0.852	23.0	1.0
4	Retuned full extension	300	59.7 [54.0, 65.1]	28.0 [23.2, 33.3]	0.838	26.7	1.3

VI. Discussion

The main empirical finding is that execution-time intervention changes the dense-traffic failure mode more reliably than reward shaping alone. Reward-only safety can reduce measured cost through non-motion, whereas the guard modifies unsafe commands while preserving route progress. The no-guard collapse provides the clearest evidence: once runtime action correction is removed, the policy loses both safety and progress.

Guard and Shield play different roles. The guard is less conservative because it applies soft TTC and speed corrections before execution, which explains its stronger success–cost balance in the formal held-out slice. The shield branch applies stronger braking fallback under severe TTC or overspeed predicates and gives the highest external-seed mean success. However, overlapping confidence intervals prevent a statistically isolated superiority claim.

The gated-risk extension should not be interpreted as the final algorithm. It is useful diagnostically because it tests whether progress-gated risk penalties improve learning, but its advantage is not consistent across evaluation blocks. Therefore, the supported contribution is the runtime safety family rather than reward shaping alone.

Limitations. The claim remains simulator-bounded. The study is limited to MetaDrive simulation and does not establish real-world deployment validity or formal verification. The runtime filter uses observable short-horizon TTC and speed cues rather than learned intent prediction, so it may fail in dense interactive scenarios where future traffic behaviour is ambiguous. The stress-density results show substantial degradation at $d = 0.20$ and $d = 0.25$. These limitations motivate adaptive guard thresholds, learned risk prediction, stronger interaction modelling, and real-time deployment studies.

VII. Conclusion

This paper introduced GuardShield-Runtime, a runtime safety architecture for progress-preserving dense-traffic RL driving. By separating action guarding, final shielding, and progress-gated risk shaping, the method identifies where safety intervention should occur in the decision pipeline. In MetaDrive, the Guard runtime mode achieves the best held-out success–cost balance at $d = 0.15$, while the Shield runtime mode obtains the highest external-seed mean success. The no-guard collapse shows that runtime action

guarding is the critical component. The results support a simulator-bounded improvement in the progress–safety trade-off and identify high-density generalisation as the main remaining challenge.

References

- [1] J. Garcia and F. Fernandez, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.
- [2] S. Gu, L. Yang, Y. Du, G. Chen, F. Walter, J. Wang, and A. Knoll, “A review of safe reinforcement learning: Methods, theory and applications,” *arXiv preprint arXiv:2205.10330*, 2022.
- [3] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *International Conference on Machine Learning*, 2017, pp. 22–31.
- [4] A. Ray, J. Achiam, and D. Amodei, “Benchmarking safe exploration in deep reinforcement learning,” *arXiv preprint arXiv:1910.01708*, 2019.
- [5] Y. Chow, O. Nachum, E. Duenez-Guzman, and M. Ghavamzadeh, “A lyapunov-based approach to safe reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2018.
- [6] C. Tessler, D. J. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” in *International Conference on Learning Representations*, 2019.
- [7] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, “Safe exploration in continuous action spaces,” *arXiv preprint arXiv:1801.08757*, 2018.
- [8] M. Alshiekh, R. Bloem, R. Ehlers, B. Koenighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *AAAI Conference on Artificial Intelligence*, 2018.
- [9] N. Fulton and A. Platzer, “Safe reinforcement learning via formal methods: Toward safe control through proof and learning,” in *AAAI Conference on Artificial Intelligence*, 2018.
- [10] K. P. Wabersich and M. N. Zeilinger, “A predictive safety filter for learning-based control of constrained nonlinear dynamical systems,” *Automatica*, vol. 129, p. 109597, 2021.
- [11] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [12] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, “End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks,” *AAAI Conference on Artificial Intelligence*, 2019.
- [13] Q. Li, Z. Peng, L. Feng, Z. Zhang, H. Xue, and B. Zhou, “Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [14] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “Carla: An open urban driving simulator,” in *Conference on Robot Learning*, 2017, pp. 1–16.
- [15] E. Leurent, “An environment for autonomous driving decision-making,” *arXiv preprint arXiv:1809.06372*, 2018.

- [16] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, “Deep reinforcement learning for autonomous driving: A survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2021.
- [17] A. Kendall, J. Hawke, D. Janz, P. Mazur, D. Reda, J.-M. Allen, V.-D. Lam, A. Bewley, and A. Shah, “Learning to drive in a day,” *International Conference on Robotics and Automation*, 2019.
- [18] B. Paden, M. Cáp, S. Z. Yong, D. Yershov, and E. Frazzoli, “A survey of motion planning and control techniques for self-driving urban vehicles,” *IEEE Transactions on Intelligent Vehicles*, vol. 1, no. 1, pp. 33–55, 2016.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [20] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International Conference on Machine Learning*, 2018, pp. 1861–1870.
- [21] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in ai safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [22] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.